

NUCLEO-G474RE

Power Electronics Project Series

PWM Generation

Project #03

Pulse Width Modulation Control

Implementation Frameworks:

STM32CubeIDE | MATLAB Embedded Coder

Document Version: 1.1

Last Updated: August 4, 2026

GitHub: <https://github.com/Anmol-G-K/NUCLEO-G474RE-PowerElectronics-Guide>

This work is licensed under the MIT License

Contents

1	Project Overview	3
1.1	Objective	3
1.2	Applications	3
1.3	Key Concepts	3
1.3.1	Pulse Width Modulation Principles	3
1.3.2	PWM Frequency and Resolution	4
1.3.3	STM32G4 Timer Peripherals	4
1.4	Learning Outcomes	4
2	STM32CubeIDE Implementation	4
2.1	Prerequisites	4
2.2	Hardware Configuration	5
2.2.1	Pin Configuration	5
2.2.2	Peripheral Configuration	5
2.3	Software Architecture	5
2.3.1	Project Structure	5
2.3.2	Initialisation Flow	5
2.3.3	Project Creation Workflow	6
2.3.4	HRTIM Setup CubeMX	6
2.4	Code Implementation	9
2.4.1	Main Initialisation	9
2.4.2	Core Functionality	9
2.5	Building and Flashing	9
2.6	Debugging and Observation	9
2.6.1	Exploration and Learning	9
2.6.2	Using the Integrated Debugger	10
3	MATLAB Embedded Coder Implementation	10
3.1	Prerequisites	10
3.2	MATLAB/Simulink Model Design	10
3.2.1	Model Architecture	10
3.2.2	Block Functionality	11
3.3	Building and Deploying the Simulink Model	11
3.3.1	Hardware Board Selection and Configuration	11
3.3.2	CubeMX Configuration	12
3.3.3	Complete Model Layout	16
3.3.4	Building and Deploying	16
4	Testing and Validation	16
4.1	Waveform Measurement	16
5	Results and Analysis	20

5.1	Project Validation	20
5.2	Observations	20
5.3	Key Findings	21
6	Troubleshooting	21
6.1	Common Issues and Solutions	21
7	Extensions and Enhancements	22
7.1	Enhancement 1: Dead-Time Insertion	22
7.2	Enhancement 2: PWM Frequency Sweep	22
7.3	Enhancement 3: Current-Mode Control	22
7.4	Further Learning	22
8	Conclusion	22
8.1	Key Takeaways	22
8.2	Next Steps	23
A	PWM Frequency Calculation Reference	25
B	Hardware Documentation	25
C	Code Listings	25

1 Project Overview

1.1 Objective

Pulse Width Modulation (PWM) is a fundamental technique in power electronics for controlling the average power delivered to a load by varying the duty cycle of a periodic waveform. This project demonstrates PWM generation using three distinct timer peripherals on the STM32G474RE: standard timers (TIM1, TIM8) and the high-resolution timer (HRTIM), showcasing different capabilities and applications.

1.2 Applications

PWM is extensively used in power electronics and general embedded systems:

Power Electronics Applications:

- DC-DC converter control and voltage regulation
- Motor speed control and direction management
- LED brightness control and dimming
- Power factor correction in AC systems
- Inverter and MOSFET gate drive signals

General Embedded Systems:

- Servo motor control
- Heater temperature regulation
- Audio signal generation
- Sensor interface timing

1.3 Key Concepts

1.3.1 Pulse Width Modulation Principles

PWM controls power delivery by switching a load on and off at high frequency. The ratio of on-time to total period is the **duty cycle**:

$$\text{Duty Cycle} = \frac{t_{\text{on}}}{T} \times 100\% \quad (1)$$

Where t_{on} is the on-time and T is the period.

The average voltage delivered to a load is proportional to the duty cycle:

$$V_{\text{avg}} = V_{\text{supply}} \times \text{Duty Cycle} \quad (2)$$

1.3.2 PWM Frequency and Resolution

PWM frequency determines switching speed. Higher frequencies reduce electromagnetic noise and ripple but increase switching losses. Frequency is calculated from the timer period:

$$f_{\text{PWM}} = \frac{f_{\text{clock}}}{(ARR + 1) \times (\text{Prescaler} + 1)} \quad (3)$$

Where ARR is the auto-reload register value.

1.3.3 STM32G4 Timer Peripherals

The STM32G4 microcontroller provides multiple timer peripherals:

- **TIM1, TIM8:** Advanced control timers with complementary PWM outputs, dead-time insertion, and break features. Ideal for motor control and MOSFET gate driving.
- **TIM2–TIM5:** General-purpose timers suitable for standard PWM applications.
- **HRTIM:** High-resolution timer with nanosecond-level timing precision, multiple independent units, and synchronized outputs. Designed for advanced power electronics applications requiring precise pulse control.

1.4 Learning Outcomes

By completing this project, you will understand:

- PWM principles and duty cycle calculation
- Configuration of standard timers (TIM1, TIM8) for PWM generation
- Complementary PWM outputs for bridge circuits
- High-resolution timer (HRTIM) for precision pulse control
- Real-time PWM parameter adjustment using external inputs
- Hardware integration of PWM signals for power circuit control

2 STM32CubeIDE Implementation

2.1 Prerequisites

Before starting this implementation, ensure you have:

- *STM32CubeIDE version 1.10 or later installed*
- *NUCLEO-G474RE development board*
- *USB cable for programming and debugging*
- *Basic familiarity with embedded C programming*
- *Completion of Project 01: Basic LED Blink*
- *Knowledge of timer peripherals and PWM concepts*

For detailed information on project creation workflow, refer to https://github.com/Anmol-G-K/NUCLEO-G474RE-PowerElectronics-Guide/tree/main/Examples/01_Basic_Led_Blink.

2.2 Hardware Configuration

2.2.1 Pin Configuration

The PWM project utilises multiple timer outputs for demonstration:

Table 1: PWM Output Pin Configuration

Pin	Function	Description
PA8	TIM1 Channel 1	PWM output with button-controlled duty cycle
PC6	TIM8 Channel 1	Complementary PWM output (65% duty)
PC5	TIM8 Channel 1N	Complementary output (logical inverse)
PA11	HRTIM Timer A Output 1	High-resolution PWM with sweep
PC13	B1 (User Button)	Input for duty cycle selection

2.2.2 Peripheral Configuration

Configuration for all three timer peripherals is performed in STM32CubeMX. For detailed configuration steps, refer to the CubeMX Configuration section below (Section 3.3.2).

2.3 Software Architecture

2.3.1 Project Structure

```

1 03_PWM_Generation/
2 |-- Core/
3 |   |-- Inc/
4 |   |   |-- main.h
5 |   |   '-- stm32g4xx_it.h
6 |   '-- Src/
7 |       |-- main.c
8 |       '-- stm32g4xx_it.c
9 |-- Drivers/
10 |   '-- STM32G4xx_HAL_Driver/
11 '-- .mxproject

```

Listing 1: Project Directory Structure

The main application logic resides in `Core/Src/main.c`. Three independent PWM generators operate simultaneously: TIM1 (user-controlled duty), TIM8 (fixed complementary), and HRTIM (dynamic sweep).

2.3.2 Initialisation Flow

PWM activation follows two key steps:

1. **Timer Configuration:** ARR, prescaler, and compare registers are configured in STM32CubeMX and auto-generated HAL initialisation functions.
2. **PWM Start:** HAL functions (`HAL_TIM_PWM_Start`, `HAL_TIMEx_PWMN_Start`, `HAL_HRTIM_WaveformOutputStart`) activate each timer's output channels.

Once started, the timers autonomously generate PWM waveforms. Software modifies duty cycle by updating the compare register (CCR) using `__HAL_TIM_SET_COMPARE()`.

2.3.3 Project Creation Workflow

For STM32CubeIDE project creation and initial setup, refer to https://github.com/Anmol-G-K/NUCLEO-G474RE-PowerElectronics-Guide/tree/main/Examples/01_Basic_Led_Blink. The workflow is identical; differences lie in timer and peripheral selection during configuration.

For the setup of the peripherals using CubeMX, refer to section 3.3.2. The workflow for TIM1 and TIM8 is identical; HRTIM is discussed below.

2.3.4 HRTIM Setup CubeMX

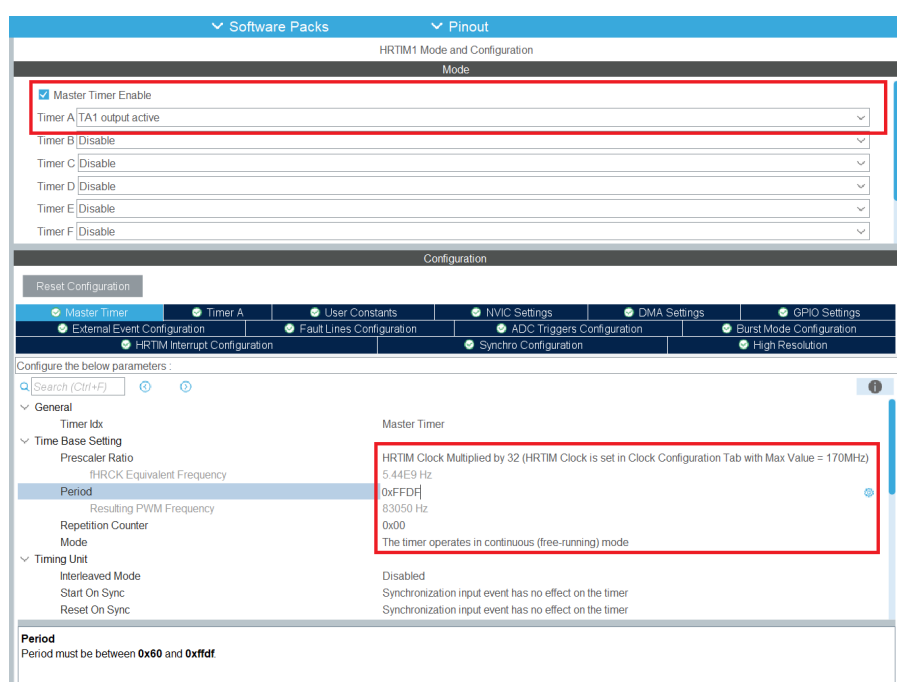


Figure 1: HRTIM master timer activation and Timer A enable.

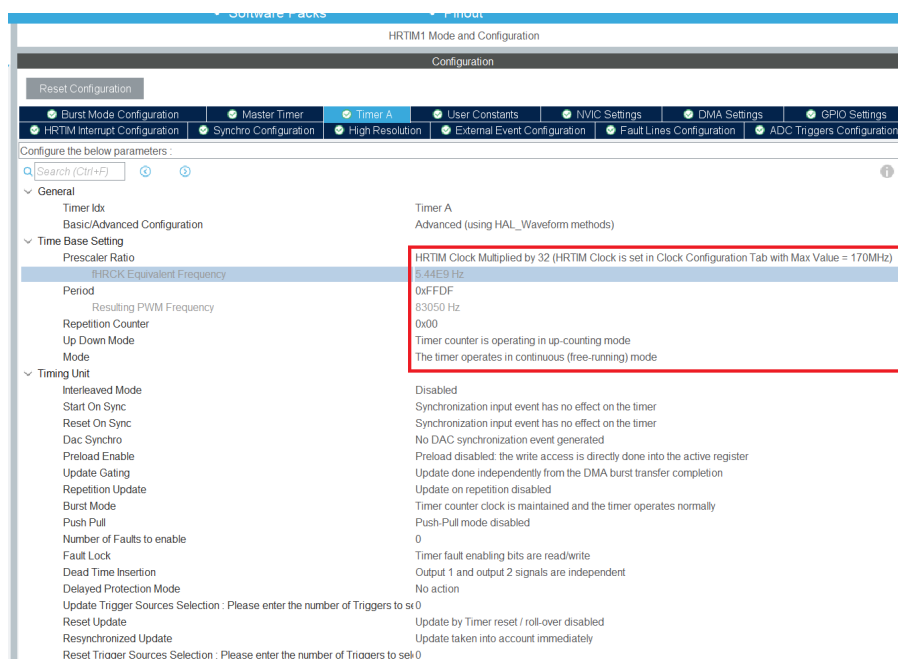


Figure 2: HRTIM Timer A frequency and period configuration (Period = 0xFFDF = 65503).

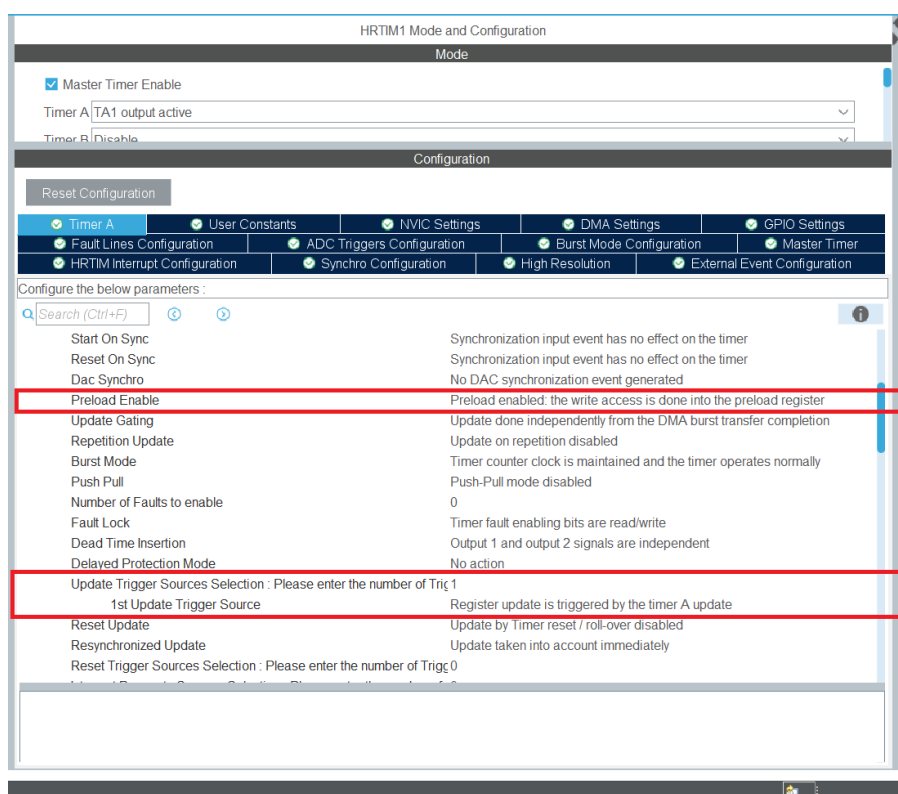


Figure 3: HRTIM Preload and Trigger Source Configuration.

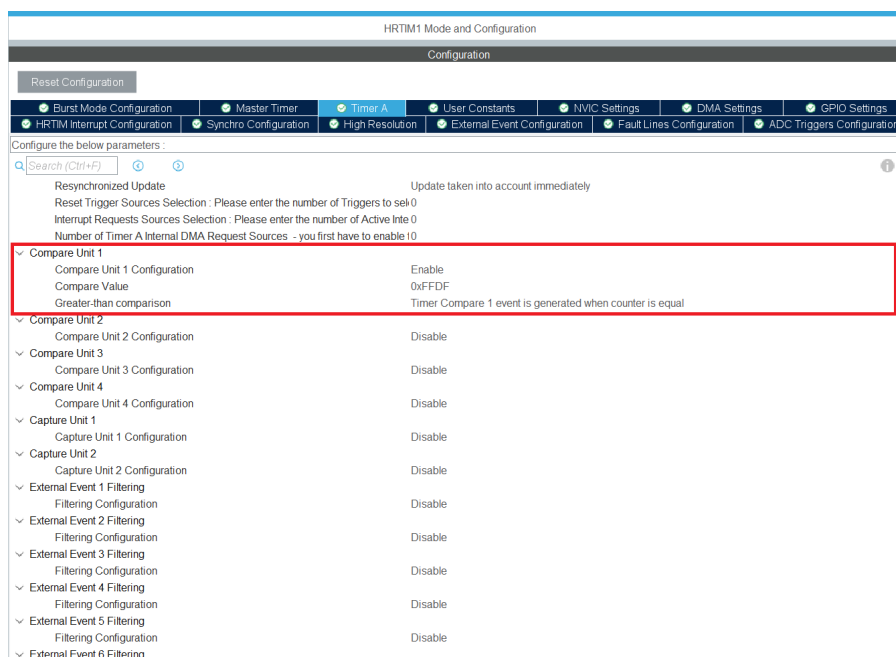


Figure 4: HRTIM output compare unit 1 enable and configuration. CMP1 value is overwritten by Simulink.

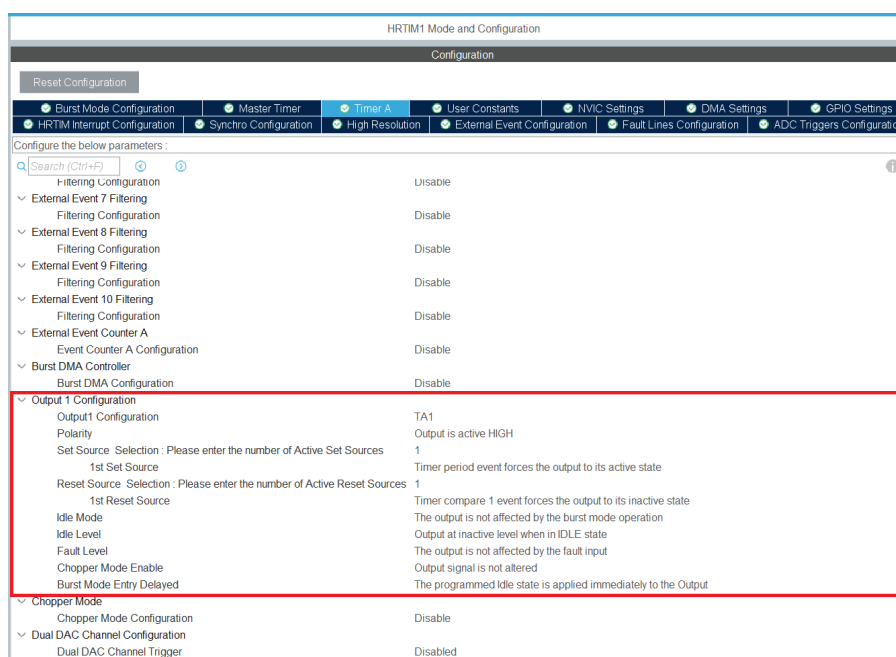


Figure 5: HRTIM set and reset signal source configuration for output waveform generation.

Note: There are slight configuration changes in the HRTIM setup between CUBE-IDE and Simulink Embedded Coder. These changes are intentional, and the resulting differences in the waveform are left for the experimenter to observe and understand.

2.4 Code Implementation

2.4.1 Main Initialisation

```

1 // Start TIM1 PWM (5 kHz, 75% default duty)
2 __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, TIM1_DUTY_75);
3 HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_1);
4
5 // Start TIM8 PWM (20 kHz, 65% / 35% complementary)
6 __HAL_TIM_SET_COMPARE(&htim8, TIM_CHANNEL_1, TIM8_DUTY_65);
7 HAL_TIM_PWM_Start(&htim8, TIM_CHANNEL_1);
8 HAL_TIMEx_PWMN_Start(&htim8, TIM_CHANNEL_1);
9
10 // Start HRTIM Timer A (pulse sweep 15% to 95%)
11 HRTIM1->sTimerxRegs[HRTIM_TIMERINDEX_TIMER_A].CMP1xR = HRTIM_CMP_MIN;
12 HAL_HRTIM_WaveformOutputStart(&hhrtim1, HRTIM_OUTPUT_TA1);
13 HAL_HRTIM_WaveformCountStart(&hhrtim1, HRTIM_TIMERID_TIMER_A);

```

Listing 2: PWM Initialisation and Startup

2.4.2 Core Functionality

```

1 // TIM1: Toggle duty 75% / 25% based on button
2 if (HAL_GPIO_ReadPin(B1_GPIO_Port, B1_Pin) == GPIO_PIN_RESET)
3     __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, TIM1_DUTY_75);
4 else
5     __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, TIM1_DUTY_25);
6
7 // HRTIM: Triangular sweep 15% -> 95% -> 15% every 50ms

```

Listing 3: PWM Parameter Update Loop

2.5 Building and Flashing

For build and flash procedures, refer to Examples/01_Basic_Led_Blink.

2.6 Debugging and Observation

2.6.1 Exploration and Learning

To deepen your understanding of PWM and timer control:

- **Review Timer Configuration:** Examine the .ioc file in STM32CubeMX to inspect ARR, prescaler, and compare register values. Understand how these values determine PWM frequency and duty cycle.
- **Study HAL Abstraction:** Review `stm32g4xx_hal_tim.c` and `stm32g4xx_hal_tim.h` to understand register operations behind HAL function calls.

- **Reference Manual Analysis:** Consult Sections 22–24 of the STM32G4 Reference Manual for detailed timer register descriptions, timing diagrams, and advanced features (dead-time, break protection).

2.6.2 Using the Integrated Debugger

To observe PWM operation in real-time:

- Set breakpoints in the main loop where duty cycle is updated
- Run in debug mode via **Debug As → STM32 C/C++ Application**
- Inspect variables `hrtim_cmp`, button state, and timer counter values
- Use oscilloscope or logic analyser connected to PWM output pins to verify waveforms

Note: The STM32CubeIDE User Manual [4] and STM32 Reference Manual [1] provide comprehensive timer documentation. External measurement equipment (oscilloscope) is essential for PWM verification.

3 MATLAB Embedded Coder Implementation

3.1 Prerequisites

Before starting the MATLAB Embedded Coder implementation, ensure you have:

- *MATLAB R2021b or later with Embedded Coder toolbox installed*
- *Embedded Coder Support Package for STMicroelectronics STM32*
- *ARM GCC Compiler configured as **GNU Tools for ARM Embedded Processors***
- *NUCLEO-G474RE development board*
- *Completion of Project 01 and 02 for workflow familiarity*

Note: The ARM GCC compiler configuration is critical for successful code generation. This must be set to **GNU Tools for ARM Embedded Processors** for STM32 compatibility.

3.2 MATLAB/Simulink Model Design

3.2.1 Model Architecture

The Simulink model implements three independent PWM systems operating simultaneously:

1. **TIM1 Control System:** Reads user button input (PC13) and dynamically switches TIM1 duty cycle between 75% (button released) and 25% (button pressed).
2. **TIM8 Complementary Output:** Generates fixed 65% PWM on TIM8 CH1 with logical complement (35%) on CH1N for bridge driver applications.
3. **HRTIM Pulse Sweep:** Implements a triangular duty cycle sweep (15% → 95% → 15%) on the high-resolution timer, demonstrating precise pulse control.

3.2.2 Block Functionality

Table 2: Simulink Block Descriptions

Block	Purpose
Digital Port Read (PC13)	Reads user button state; drives conditional logic for TIM1 duty selection
PWM Output (TIM1)	Generates 5 kHz PWM on PA8 with duty 75% or 25% based on button
PWM Output (TIM8)	Generates 20 kHz PWM on PC6 (CH1) and PC5 (CH1N) at 65% / 35%
Ramp Signal	Linear ramp function for triangular sweep generation
PWM Output (HRTIM)	High-resolution PWM on PA11 with dynamic duty driven by ramp signal
Scope	Visualises all three PWM outputs and button input in real-time

3.3 Building and Deploying the Simulink Model

3.3.1 Hardware Board Selection and Configuration

For hardware selection, STM32CubeMX configuration, and compiler setup, refer to https://github.com/Anmol-G-K/NUCLEO-G474RE-PowerElectronics-Guide/tree/main/Examples/01_Basic_Led_Blink. The process is identical to previous projects.

3.3.2 CubeMX Configuration

Configuration of the three timers in STM32CubeMX is critical for correct PWM operation. The following images illustrate the necessary settings:

HRTIM Configuration:

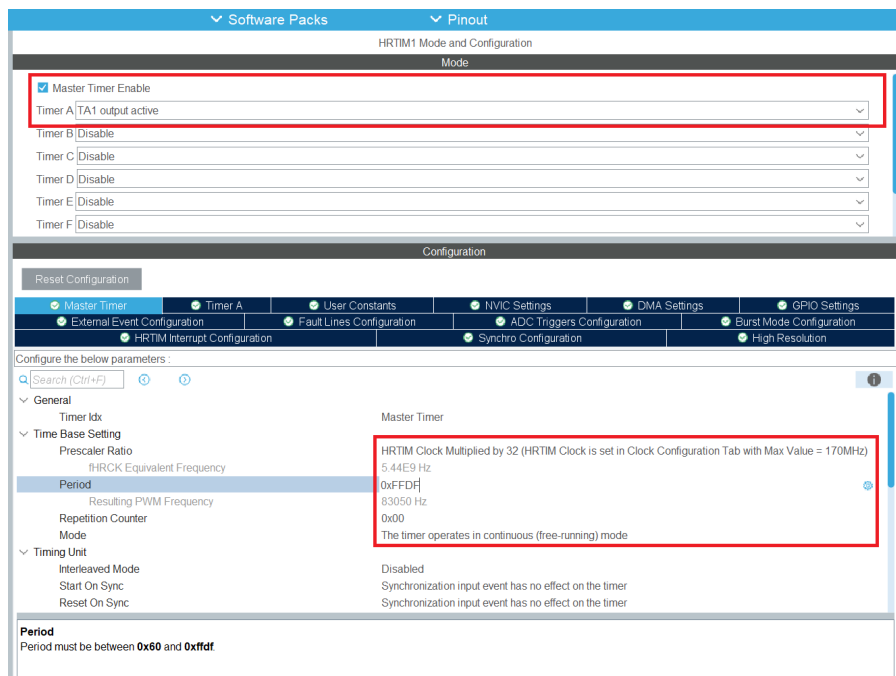


Figure 6: HRTIM master timer activation and Timer A enable.

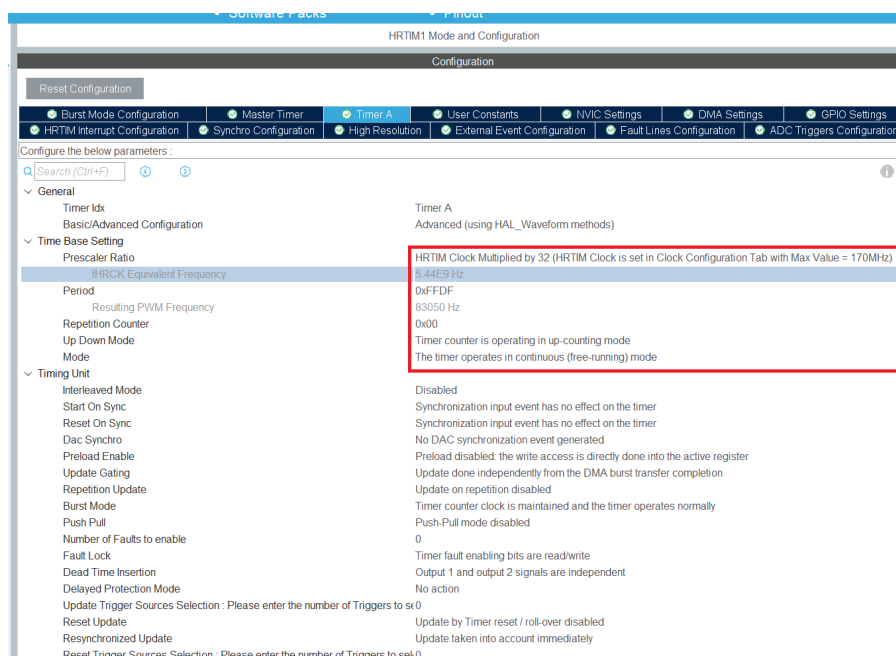


Figure 7: HRTIM Timer A frequency and period configuration (Period = 0xFFDF = 65503).

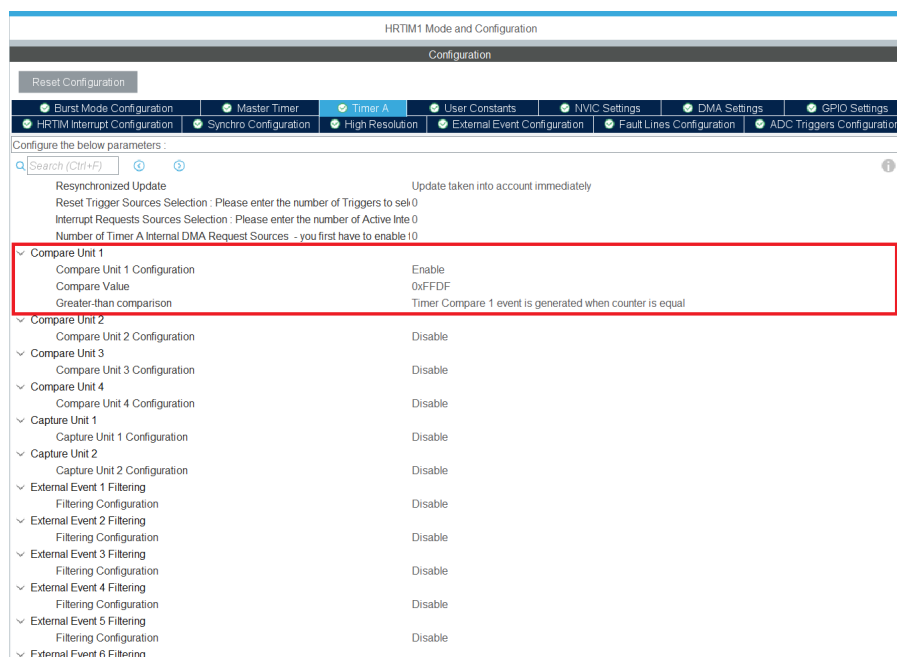


Figure 8: HRTIM output compare unit 1 enable and configuration. CMP1 value is overwritten by Simulink.

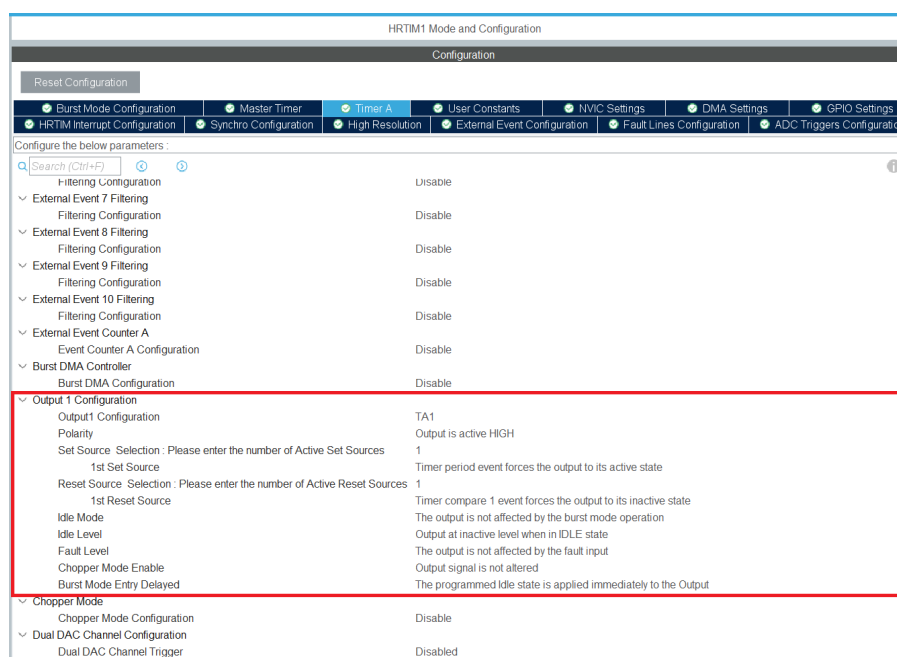


Figure 9: HRTIM set and reset signal source configuration for output waveform generation.

Note: Minor intentional HRTIM configuration differences exist between CUBE-IDE and Simulink Embedded Coder; resulting waveform variations are left for the user to observe.

TIM1 Configuration:

TIM1 Mode and Configuration

Mode

Slave Mode: Disable

Trigger Source: Disable

Clock Source: Internal Clock

Channel1: PWM Generation CH1

Channel2: Disable

Channel3: Disable

Channel4: Disable

Configuration

Reset Configuration

Parameter Settings User Constants NVIC Settings DMA Settings GPIO Settings

Configure the below parameters:

Search (Ctrl+F)

Counter Settings

Prescaler (PSC - 16 bits value): 0

Counter Mode: Up

Dithering: Disable

Counter Period (AutoReload Register - 16 bits value): 33999

Internal Clock Division (CKD): No Division

Repetition Counter (RCR - 16 bits value): 0

auto-reload preload: Enable

Trigger Output (TRGO) Parameters

Master/Slave Mode (MSM bit): Disable (Trigger input effect not delayed)

Trigger Event Selection TRGO: Reset (UG bit from TIMx_EGR)

Trigger Event Selection TRGO2: Reset (UG bit from TIMx_EGR)

Break And Dead Time management - BRK Configuration

BRK State: Disable

BRK Polarity: High

Figure 10: TIM1 frequency configuration. $ARR = 33999$ yields 5 kHz PWM.

Software Packs Pinout

TIM1 Mode and Configuration

Mode

Configuration

Reset Configuration

Parameter Settings User Constants NVIC Settings DMA Settings GPIO Settings

Configure the below parameters:

Search (Ctrl+F)

BRK2 Polarity: High

BRK2 Filter (4 bits value): 0

BRK2 Sources Configuration

- Digital Input: Disable

- COMP1: Disable

- COMP2: Disable

- COMP3: Disable

- COMP4: Disable

- COMP5: Disable

- COMP6: Disable

- COMP7: Disable

Break And Dead Time management - Output Configuration

Automatic Output State: Disable

Off State Selection for Run Mode (OSSR): Disable

Off State Selection for Idle Mode (OSSI): Disable

Lock Configuration: Off

Clear Input

Clear Input Source: Disable

PWM Generation Channel 1

Mode: PWM mode 1

Pulse (16 bits value): 0

Output compare preload: Enable

Fast Mode: Disable

CH Polarity: High

CH Idle State: Reset

Figure 11: TIM1 CH1 configured for PWM generation in active mode.

TIM8 Configuration:

TIM8 Mode and Configuration

Mode

Slave Mode: Disable

Trigger Source: Disable

Clock Source: Internal Clock

Channel1: PWM Generation CH1 CH1N

Channel2: Disable

Channel3: Disable

Channel4: Disable

Configuration

Reset Configuration

Parameter Settings | User Constants | NVIC Settings | DMA Settings | GPIO Settings

Configure the below parameters:

Search (Ctrl+F)

Counter Settings

Prescaler (PSC - 16 bits value): 0

Counter Mode: Up

Dithering: Disable

Counter Period (AutoReload Register - 16 bits value): 8499

Internal Clock Division (CKD): No Division

Repetition Counter (RCR - 16 bits value): 0

auto-reload preload: Enable

Trigger Output (TRGO) Parameters

Master/Slave Mode (MSM bit): Disable (Trigger input effect not delayed)

Trigger Event Selection TRGO: Reset (UG bit from TIMx_EGR)

Trigger Event Selection TRGO2: Reset (UG bit from TIMx_EGR)

Break And Dead Time management - BRK Configuration

BRK State: Disable

BRK Polarity: High

Figure 12: TIM8 frequency configuration. $ARR = 8499$ yields 20 kHz PWM.

TIM8 Mode and Configuration

Mode

Slave Mode: Disable

Configuration

Reset Configuration

Parameter Settings | User Constants | NVIC Settings | DMA Settings | GPIO Settings

Configure the below parameters:

Search (Ctrl+F)

Break And Dead Time management - Output Configuration

Automatic Output State: Disable

Off State Selection for Run Mode (OSSR): Disable

Off State Selection for Idle Mode (OSSR): Disable

Lock Configuration: Off

DeadTime Preload: Disable

DeadTime: 0

Asymmetrical DeadTime: Disable

Falling Dead Time: 0

Clear Input

Clear Input Source: Disable

PWM Generation Channel 1 and 1N

Mode: PWM mode 1

Pulse (16 bits value): 0

Output compare preload: Enable

Fast Mode: Disable

CH Polarity: High

CHN Polarity: High

CH Idle State: Reset

CHN Idle State: Reset

Figure 13: TIM8 CH1 and CH1N configured for complementary PWM generation.

3.3.3 Complete Model Layout

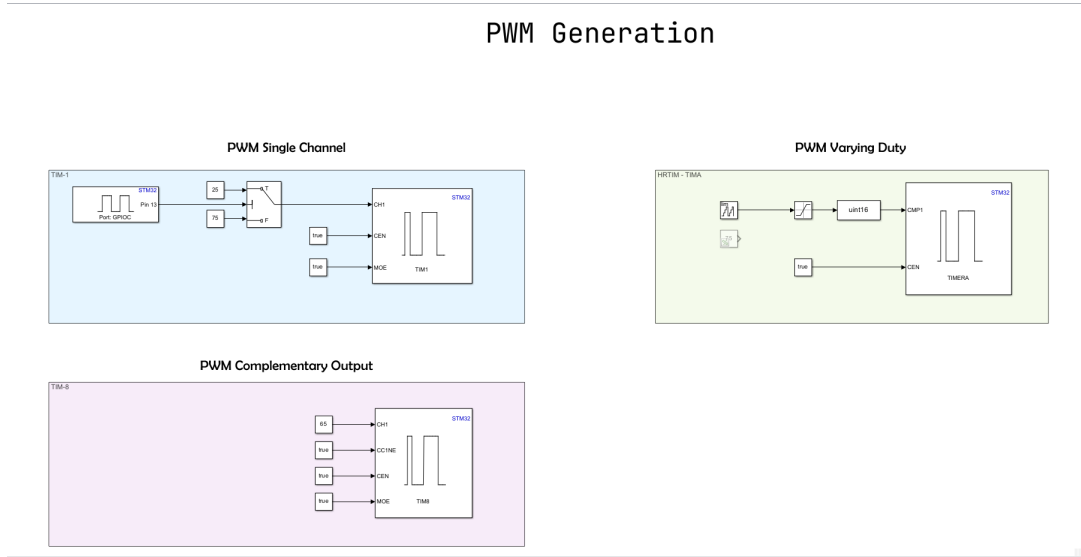


Figure 14: Complete Simulink model architecture showing three PWM systems. For the HRTIM block, a constant duty ratio input can also be used; this option is included but commented out in the Simulink model.

3.3.4 Building and Deploying

For detailed build, deployment, and execution procedures, refer to https://github.com/Anmol-G-K/NUCLEO-G474RE-PowerElectronics-Guide/tree/main/Examples/01_Basic_Led_Blink. The workflow is identical to previous projects: configure hardware board, set compiler, then build and deploy.

4 Testing and Validation

4.1 Waveform Measurement

PWM operation is verified by observing output waveforms on an oscilloscope or logic analyser. All three timer outputs should generate the configured waveforms simultaneously.

Category	Tool	Notes
Oscilloscope	Hantek 6022BL + OpenHantek	Used for waveform verification
Oscilloscope	Any 2-channel oscilloscope	Recommended minimum requirement
Logic Analyzer	USB Logic Analyzer (24MHz, 8CH)	Basic alternative (not recommended)
Software	PulseView (Sigrok),	Open-source logic analyzer software
Software	Saleae Logic Logic	Proprietary, user-friendly interface

Table 3: Tools used for PWM waveform verification

STM32CubeIDE Implementation Waveforms:

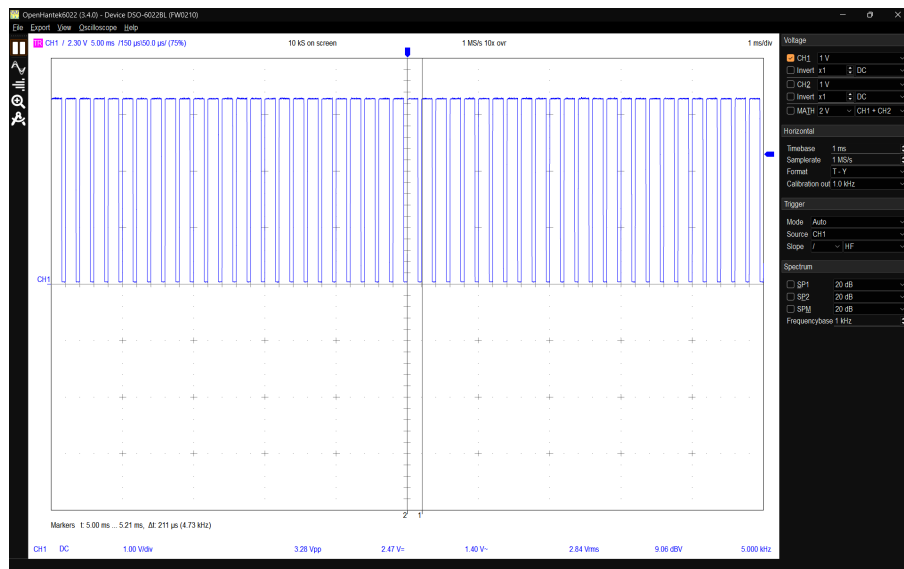


Figure 15: TIM1 output at 75% duty cycle (button unpressed). Frequency: 5 kHz.

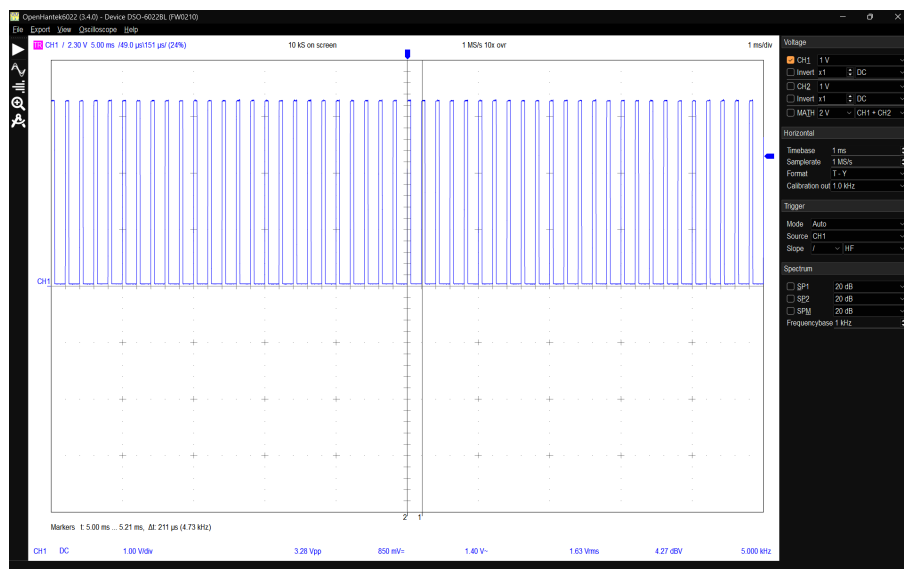


Figure 16: TIM1 output at 25% duty cycle (button pressed). Same frequency, different duty.

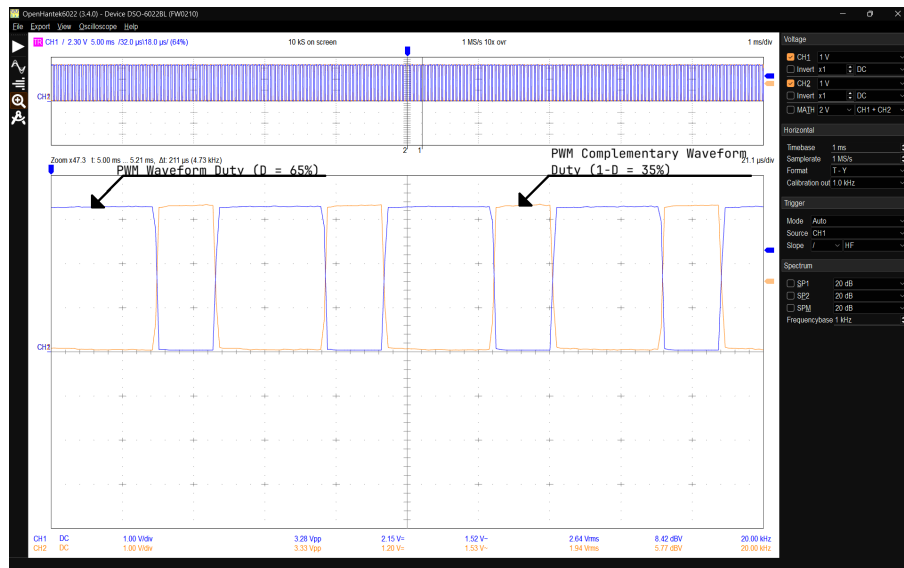


Figure 17: TIM8 complementary outputs: CH1 (65%, top) and CH1N (35%, bottom) are logical complements.

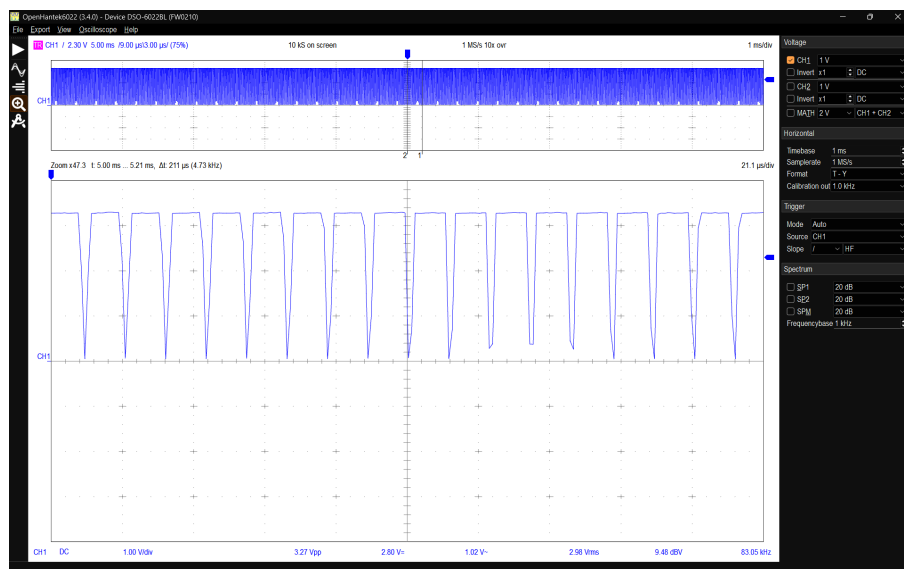


Figure 18: HRTIM output during sweep phase showing 75% duty cycle.

MATLAB Embedded Coder Implementation Waveforms:

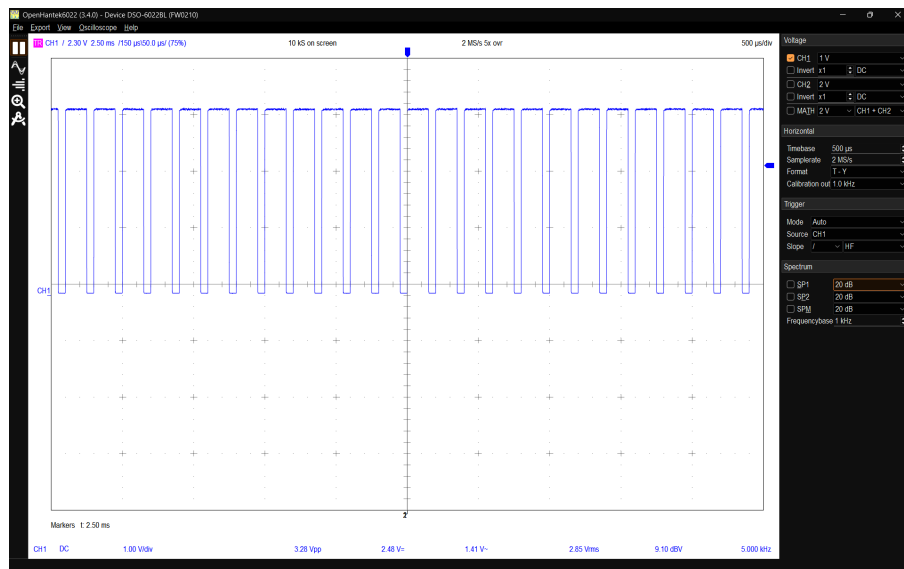


Figure 19: Simulink Embedded Coder: TIM1 output at 75% duty. Matches CubeIDE implementation.

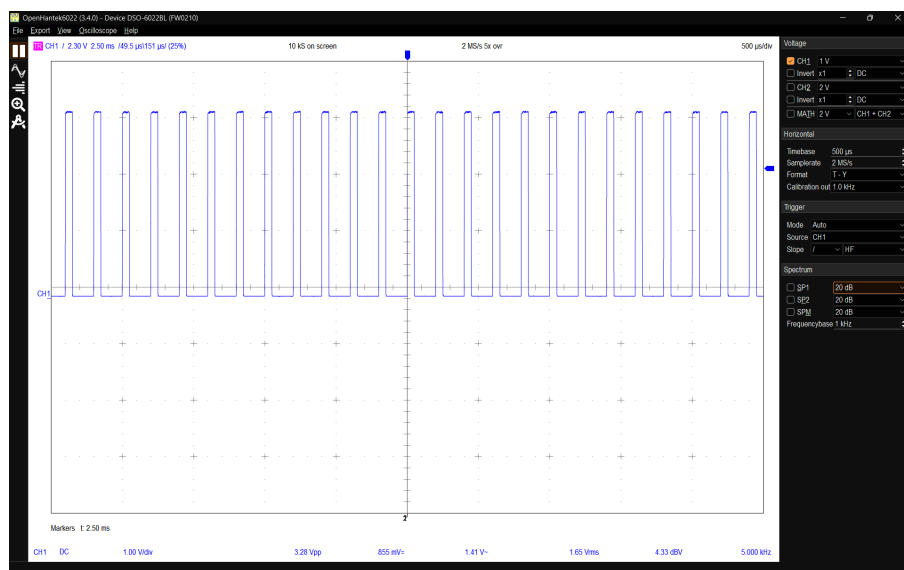


Figure 20: Simulink Embedded Coder: TIM1 output at 25% duty (button pressed).

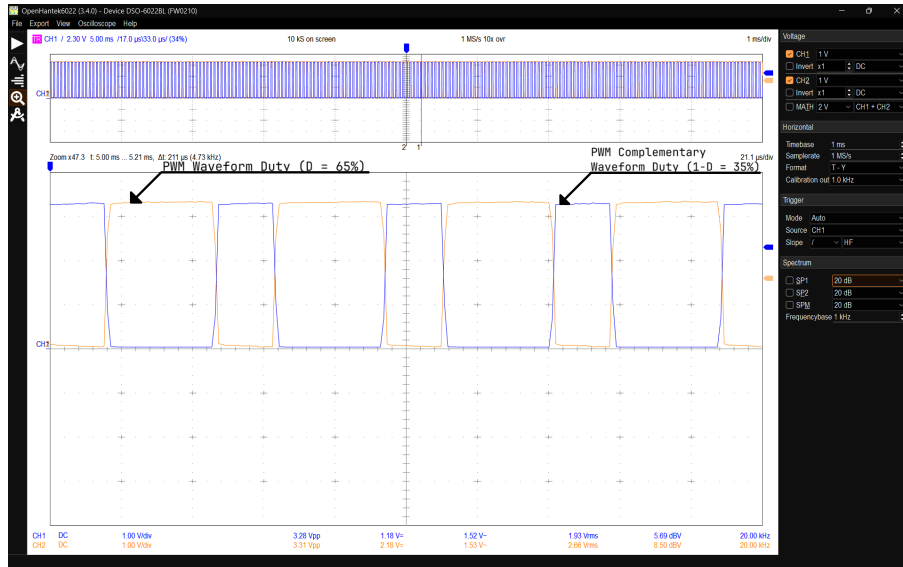


Figure 21: Simulink Embedded Coder: TIM8 complementary outputs at 65% / 35%.

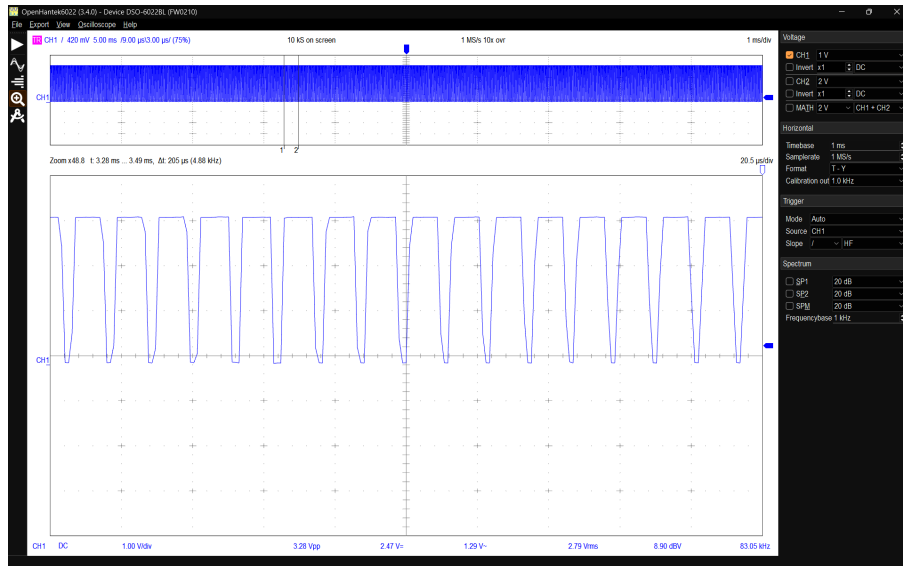


Figure 22: Simulink Embedded Coder: HRTIM output during sweep at 75% duty.

5 Results and Analysis

5.1 Project Validation

Both STM32CubeIDE and MATLAB Embedded Coder implementations produce equivalent PWM waveforms. Frequency, duty cycle, and switching characteristics closely match between implementations, confirming correct functionality.

5.2 Observations

1. **TIM1 Button Control:** Duty cycle switches cleanly between 75% and 25% on button press, with no glitches or transients.

2. **TIM8 Complementary Output:** CH1 and CH1N maintain logical complement relationship, essential for half-bridge gate drive applications. *Note: Dead-time insertion is currently disabled. For robust bridge operation, a dead-time value should be added to the TIM8 break configuration to prevent cross-conduction. This can be implemented by writing to the BDTR register.*
3. **HRTIM Sweep:** Pulse width sweeps smoothly across the 15%–95% range, demonstrating high-resolution timer precision. Shadow register preload prevents glitches during updates.

5.3 Key Findings

Both implementations successfully demonstrate PWM generation at different frequencies and with different characteristics. The slight variations in HRTIM configuration between CubeIDE and MATLAB highlight the trade-offs between direct register control and model-based abstraction.

6 Troubleshooting

6.1 Common Issues and Solutions

Table 4: PWM Troubleshooting Guide

Issue	Solution
No PWM output on oscilloscope	Verify pin configuration in CubeMX; check that HAL_TIM_PWM_Start is called; confirm oscilloscope probe connected to correct pin
Wrong PWM frequency	Check ARR and prescaler values; verify system clock frequency; recalculate using frequency formula
Duty cycle not changing with button	Inspect button GPIO configuration; verify button input logic (active HIGH/LOW); check that compare register update occurs in main loop
HRTIM waveform glitches	HRTIM configuration requires careful setup. Verify PreloadEnable and UpdateTrigger settings in CubeMX; check shadow register transfers
Complementary outputs not correct	Verify CH1N polarity setting; confirm both PWM_Start and PWMN_Start are called; inspect break and dead-time configuration

Note: HRTIM configuration is complex and sensitive to incorrect parameter settings. Careful attention to the configuration images and reference manual is essential for successful operation.

7 Extensions and Enhancements

7.1 Enhancement 1: Dead-Time Insertion

Implement dead-time on complementary TIM8 outputs to prevent cross-conduction in bridge circuits. This requires configuring the BDTR register or CubeMX dead-time parameter.

7.2 Enhancement 2: PWM Frequency Sweep

Extend HRTIM to sweep frequency (period) rather than just duty cycle, demonstrating full dynamic control capabilities.

7.3 Enhancement 3: Current-Mode Control

Integrate ADC feedback from Project 02 to implement current-mode PWM control, creating a closed-loop power electronics system.

7.4 Further Learning

Advanced PWM topics include:

- Space vector PWM (SVPWM) for three-phase inverters
- Interleaved PWM for multi-phase converters
- Resonant PWM for soft-switching applications
- Dithering and noise-shaping techniques for improved linearity

8 Conclusion

This project has demonstrated PWM generation using three distinct timer peripherals on the STM32G474RE. Standard timers (TIM1, TIM8) provide robust PWM at kilohertz frequencies suitable for most applications, whilst the high-resolution timer (HRTIM) offers nanosecond-level precision for advanced power electronics.

8.1 Key Takeaways

- PWM is fundamental to power electronic control and energy conversion
- STM32G4 offers multiple timer options tailored to different application requirements
- Duty cycle, frequency, and complementary outputs are essential PWM parameters
- Both traditional C-based and model-based design approaches yield equivalent results
- External measurement equipment is necessary to verify PWM operation

8.2 Next Steps

Subsequent projects build upon PWM fundamentals:

- Motor control with closed-loop feedback (Project 04)
- MOSFET gate drive circuits and dead-time management
- Power converter design and optimisation
- Advanced control algorithms (PID, model predictive control)

References

References

- [1] STMicroelectronics, *STM32G4 Series Advanced ARM-based 32-bit MCUs Reference Manual*, RM0440.
- [2] STMicroelectronics, *STM32G474RE Datasheet*.
- [3] STMicroelectronics, *STM32G4 NUCLEO-64 Boards User Manual*, UM2505.
- [4] STMicroelectronics, *STM32CubeIDE Integrated Development Environment User Manual*, DM00629855.
- [5] STMicroelectronics, *Description of STM32G4 HAL and Low-Layer Drivers*, UM2570.
- [6] STMicroelectronics, *HRTIM Cookbook*, AN4539.
- [7] MathWorks, *Embedded Coder Documentation*.
- [8] MathWorks, *Embedded Coder Support Package for STMicroelectronics STM32*.

A PWM Frequency Calculation Reference

Standard Timer PWM Frequency:

$$f_{\text{PWM}} = \frac{f_{\text{clock}}}{(ARR + 1) \times (\text{Prescaler} + 1)} \quad (4)$$

For TIM1 (5 kHz): $f = \frac{170 \text{ MHz}}{34000 \times 1} = 5 \text{ kHz}$

For TIM8 (20 kHz): $f = \frac{170 \text{ MHz}}{8500 \times 1} = 20 \text{ kHz}$

Duty Cycle from Compare Register:

$$\text{Duty} = \frac{\text{CCR}}{ARR + 1} \times 100\% \quad (5)$$

B Hardware Documentation

Refer to the official NUCLEO-G474RE schematic and STM32G474RE datasheet for complete pin definitions, electrical specifications, and timer capabilities.

C Code Listings

Complete project source code is available in the GitHub repository: https://github.com/Anmol-G-K/NUCLEO-G474RE-PowerElectronics-Guide/tree/main/Examples/03_PWM_Generation.

Author

Anmol Govindarajapuram Krishnan

GitHub: <https://github.com/Anmol-G-K>

Email: cb.en.u4eee23103@cb.students.amrita.edu

Acknowledgments

We acknowledge the following for their contributions and support:

- **STMicroelectronics** for providing comprehensive microcontroller documentation and development tools
- **MathWorks** for excellent MATLAB and Embedded Coder resources
- The embedded systems and power electronics community for innovation and knowledge sharing
- Contributors and reviewers providing valuable feedback

This project is licensed under the MIT License.

For updates and additional resources:

<https://github.com/Anmol-G-K/NUCLEO-G474RE-PowerElectronics-Guide>